

Word Spy AI — System Write-up

Overview

Word Spy AI is a Codenames-inspired single-player web game. The player can try to find all 8 hidden target words on a 5x5 word board while avoiding neutral words and one danger word. The player starts with 500 points and can either guess directly or spend points to ask an AI spymaster for a clue. Each AI clue becomes more expensive over time, so the player must balance risk and reward: guessing without help can build a higher score streak, but using AI clues can reduce uncertainty. The game ends when the player finds all target words, reveals the danger word, or drops below 0 points.

System Architecture

The system is built with Next.js, TypeScript, and Tailwind CSS. The frontend manages the game state, including the board, score, revealed cards, correct streak, clue history, and win/loss conditions. The backend uses a Next.js API route to generate AI clues. When the user requests a clue, the frontend sends the hidden board state and current game context to the backend. The backend filters this board into remaining target, neutral, and danger words before prompting the LLM, along with the current score, clue cost, neutral hits, correct streak, and previous clue history.

How AI Is Used

The LLM acts as the spymaster: it sees the hidden target, neutral, and danger words, then generates a word-association clue and a count. The model decides whether to give a conservative or ambitious clue based on the current game pressure. If the clue is expensive and there are safe connections between multiple target words, it may provide a higher-count clue. If previous clues were not useful or led to bad guesses, the model is instructed to use a different angle and avoid repeating earlier clues.

How Data Is Used

Data is used through a word corpus containing hundreds of common nouns across different categories. Each new game randomly samples 25 unique words from this corpus and assigns hidden roles: 8 target words, 16 neutral words, and 1 danger word. This makes the board generation data-driven and gives the game replayability.

Key Design Decisions

Several design decisions were made to adapt the original two-team pressure of Codenames into a single-player experience. Instead of using an opposing team, Word Spy AI uses a score economy: AI clues become increasingly expensive, correct guesses build a streak bonus, neutral guesses have punishments, and the danger word causes an instant loss. This creates tension without requiring multiplayer. The game also records every AI clue and the guesses made after it, allowing the player to review how each clue influenced their decisions. At the end of the game, all word roles are revealed for post-game review.

Technically, the game keeps state on the client side because each session is lightweight and does not require login or persistence. Each card stores its word, hidden role, and revealed status. The backend validates AI output by checking for board-word reuse and clue repetition.

Tools Used

Key tools used include Next.js for the web app and API route, TypeScript for safer game state modeling, Tailwind CSS for responsive styling, the OpenAI API for clue generation, and Vercel for deployment. The system also includes fallback clue generation so the game remains playable if the AI API is unavailable.